



```
def main():  
    print("🚀 Iniciando Playbook Agêntico...")  
    # Camada 1: Configuração e Inicialização  
    # Camada 2: Análise e Planejamento  
    # Camada 3: Execução e Monitoramento  
    # Camada 4: Avaliação e Iteração  
    print("📋 Passo 1: Definição de Objetivos e Contexto")  
    # ...  
    print("📋 Passo 2: Análise de Dados e Identificação de Oportunidades")  
    # ...  
    print("📋 Passo 3: Planejamento de Campanha e Alocação de Recursos")  
    # ...  
    print("📋 Passo 4: Execução da Campanha e Monitoramento em Tempo Real")  
    # ...  
    print("📋 Passo 5: Avaliação de Resultados e Iteração para Melhorias")  
    # ...  
    print("🎉 Playbook concluído com sucesso!")  
    return True  
  
if __name__ == "__main__":  
    main()
```

— A CENTRAL DE COMANDO AUTÔNOMA —

PLAYBOOK — O TRATADO DAS 4 CAMADAS DA FÁBRICA AGÊNTICA

◆ PARA INICIANTES

16 capítulos

projetos práticos

AUTOR

Heverton Eduardo Peres

Especialista em Marketing e Desenvolvimento de Soluções

16
CAPÍTULOS

70+
PÁGINAS

∞
PROJETOS

Objetivo do Material

O leitor é o comandante de uma central de controle onde quatro painéis operacionais (TELA, HARNESS, LLM, TOOLS) governam agentes de IA como se fossem equipes especializadas.

Obra-mãe: tratado-4-camadas

Passos práticos: 20

Você é: Engenheiro de Sistemas Autônomos

Sumário

Objetivo do Material	4
Como usar este playbook	5
Mapa dos Estágios	6
Passos Práticos	7
Passo 1 — O Contexto Real de Origem: O Projeto Arsenal Open Source	7
Passo 2 — O Dicionário do Iniciante: Glossário Descomplicado	9
Passo 3 — A Crise do Desenvolvimento com IA: Os 4 Problemas Catastróficos	10
Passo 4 — Visão Geral das 4 Camadas: A Arquitetura Completa	11
Passo 5 — Os 3 Princípios Universais da TELA	12
Passo 6 — A Constituição Mestre: As 18 Regras Sagradas	13
Passo 7 — O Motor de Economia Severa de Tokens	14
Passo 8 — Implementação e Réplica da Camada 1	15
Passo 9 — Os 3 Princípios Universais do HARNESS	17
Passo 10 — Configuração Industrial: Circuit Breakers e Sandbox	18
Passo 11 — O Guarda-Costas do Git: Pre-Commit com 6 Gates	20
Passo 12 — Implementação e Réplica da Camada 2	21
Passo 13 — Os 3 Princípios Universais do LLM	23
Passo 14 — A Matriz de 3 Tiers de Modelos	24
Passo 15 — Contratos Tipados e Registro Declarativo	26
Passo 16 — Implementação e Réplica da Camada 3	27
Passo 17 — Os 3 Princípios Universais de TOOLS	29
Passo 18 — O Banco de Estado Persistente: SQLite e a Esteira	30
Passo 19 — Servidores MCP e a Usina de Scripts Determinísticos	32
Passo 20 — O Super-Auditor e o Manual de Montagem Universal	33
Checklist Mestre	36

Objetivo do Material

Apresentar a crise do desenvolvimento com IA e a solução das 4 Camadas.

Como usar este playbook

Você é o **Engenheiro de Sistemas Autônomos**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Central de comando	1, 2, 3, 4
2	Painel operacional	5, 6, 7, 8
3	Circuito de segurança	9, 10, 11, 12
4	Roteador cognitivo	13, 14, 15, 16
5	Usina determinística	17, 18, 19, 20

Passos Práticos

Passo 1 — O Contexto Real de Origem: O Projeto Arsenal Open Source

Estágio: Central de comando · **Origem:** Cap. 1 — O Contexto Real de Origem: O Projeto Arsenal Open Source

① Objetivo do passo

Apresentar o projeto Arsenal como campo de batalha real e a matriz de transposição universal.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- (a completar)

④ Execução

4.1 Estrutura de Diretórios de uma Estação Agêntica Profissional

```
meu-projeto-agentico/
├── .governance/                # Camada 1: Diretivas, Regras e Contexto
│   ├── CONSTITUTION.md        # As 18 Regras Sagradas invioláveis
│   └── skills/                 # Habilidades carregadas sob demanda (Lazy Loading)
├── .harness/                   # Camada 2: Proteção, Hooks e Disjuntores
│   ├── settings.json          # Limites de turnos, timeouts e comandos bloqueados
│   └── hooks/                  # Pre-commit e interceptores de ferramentas
├── .router/                    # Camada 3: Motor Cognitivo e Roteamento
│   └── models_config.json      # Matriz de 3 Tiers (Flash, Standard, Reasoning)
├── .tools/                     # Camada 4: Servidores MCP e Banco de Estado
│   ├── state_tracker.db       # SQLite para persistência de tarefas
│   └── mcp_servers/           # Protocolo Model Context Protocol
├── CLAUDE.md                   # Ponto de entrada invariante para agentes
└── README.md                   # Documentação executiva
```

4.2 Script de Verificação de Prontidão da Estação (Pre-Flight Check)

```
#!/usr/bin/env python3
# preflight_check.py – Validador de integridade da estação agêntica
import os
import sys
```

```

from pathlib import Path

def verificar_estacao():
    print("=== [PRE-FLIGHT CHECK] Verificando Central de Comando Agêntica ===")
    erros = []

    # 1. Checagem de Governança (Camada 1)
    if not Path("CLAUDE.md").exists() and not Path(".governance").exists():
        erros.append("[Camada 1] Ausência de arquivo de governança invariante (CLAUDE.md).")
    else:
        print("[OK] Camada 1 (Contexto & Diretivas): Ativa e Invariante.")

    # 2. Checagem de Harness e Git (Camada 2)
    if not Path(".git").exists():
        erros.append("[Camada 2] Repositório Git não inicializado para versionamento.")
    else:
        print("[OK] Camada 2 (Harness & Execução): Controle de versão ativo.")

    # 3. Resumo da Verificação
    if erros:
        print("
[FALHA] Alertas de Prontidão detectados:")
        for err in erros:
            print(f"    -> {err}")
        return False

    print("
[SUCESSO] Central de Comando 100% operacional para o Engenheiro Agêntico!")
    return True

if __name__ == "__main__":
    if not verificar_estacao():
        sys.exit(1)

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Verifique se seu projeto possui um arquivo de governança invariante (CLAUDE.md ou equivalente — se não existir, crie um com 3 regras mínimas)
- ☐ Escreva uma lista das 4 dores que você já enfrentou ao usar IA para programar (custo, amnésia, alucinação, lock-in)
- ☐ Documente o seu fluxo atual de interação com agentes de IA e identifique ao menos 1 ponto onde a Metriz de Transposição Universal poderia ser aplicada
- ☐ Crie um esboço em papel ou Markdown de como sua Central de Comando seria organizada (4 camadas)

⑦ Armadilhas

- (a completar)

Passo 2 — O Dicionário do Iniciante: Glossário Descomplicado

Estágio: Central de comando · **Origem:** Cap. 2 — O Dicionário do Iniciante: Glossário Descomplicado

① Objetivo do passo

Garantir que qualquer pessoa domine os conceitos fundamentais de IA e engenharia.

② Pré-requisito

Passo 1 concluído

③ Entregas

- `CLAUDE.md`
- `settings.json`

④ Execução

(a completar)

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Defina em suas próprias palavras o que são LLM, MCP, Token e SQLite — escreva como se explicasse para um colega de trabalho leigo
- ☐ Crie um glossário pessoal com pelo menos 10 termos do capítulo que você mais usa ou pretende usar no dia a dia
- ☐ Identifique 3 termos que ainda lhe causam confusão e pesquise cada um até dominar o conceito
- ☐ Explique para alguém leigo o que é um “agente de IA” usando apenas analogias do mundo real

⑦ Armadilhas

- (a completar)

Passo 3 — A Crise do Desenvolvimento com IA: Os 4 Problemas Catastróficos

Estágio: Central de comando · **Origem:** Cap. 3 — A Crise do Desenvolvimento com IA: Os 4 Problemas Catastróficos

① Objetivo do passo

Detalhar cada um dos 4 problemas com exemplos reais e dados de mercado.

② Pré-requisito

Passo 2 concluído

③ Entregas

- (a completar)

④ Execução

(a completar)

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Analise uma conversa recente com IA e identifique se houve amnésia (Lost in the Middle) — o agente esqueceu algo que foi dito anteriormente?
- ☐ Calcule aproximadamente quanto gastou em tokens na última semana de uso de IA e compare com o que gastaria com Invariância de Prefixo ativa
- ☐ Liste 3 códigos gerados por IA que precisaram de correção manual depois de o agente declarar “sucesso”
- ☐ Documente 1 caso concreto de alucinação que já presenciou: o agente afirmou que algo estava pronto, mas não estava

⑦ Armadilhas

- (a completar)

Passo 4 — Visão Geral das 4 Camadas: A Arquitetura Completa

Estágio: Central de comando · **Origem:** Cap. 4 — Visão Geral das 4 Camadas: A Arquitetura Completa

① Objetivo do passo

Apresentar o modelo das 4 Camadas como solução definitiva.

② Pré-requisito

Passo 3 concluído

③ Entregas

- (a completar)

④ Execução

O Contrato de Passagem entre Camadas

```
{
  "transacao_agentica": {
    "camada_1_contexto": {
      "invariancia_prefixo": true,
      "regras_ativas": 18,
      "densidade_tokens": "shannon_max"
    },
    "camada_2_harness": {
      "circuit_breaker_limite_turnos": 15,
      "sandbox_isolada": true,
      "gate_precommit_ativo": true
    },
    "camada_3_cognitivo": {
      "modelo_selecionado": "claude-3-7-sonnet",
      "tier": 2,
      "contrato_schema": "StructuredOutput_v1"
    },
    "camada_4_ferramentas": {
      "protocolo": "mcp_json_rpc",
      "persistencia": "sqlite_wal",
      "exit_code_esperado": 0
    }
  }
}
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Esboce em papel ou Markdown como as 4 camadas se conectam no seu projeto atual — use o modelo da Fórmula 1 como referência visual
- ☐ Identifique em qual das 4 camadas seu projeto está mais fraco hoje e Justifique por quê
- ☐ Crie um plano de implementação para a Camada 1 (Contexto & Diretivas) com pelo menos 3 ações concretas
- ☐ Mapeie as ferramentas que você usa hoje e classifique cada uma em qual camada ela opera

⑦ Armadilhas

- (a completar)

Passo 5 — Os 3 Princípios Universais da TELA

Estágio: Painel operacional · **Origem:** Cap. 5 — Os 3 Princípios Universais da TELA

① Objetivo do passo

Ensinar os fundamentos de Prompt & Context Engineering.

② Pré-requisito

Passo 4 concluído

③ Entregas

- `CLAUDE.md`

④ Execução

Exemplo Real: O Cabeçalho de Governança Invariante (`CLAUDE.md`)

```
<!-- INÍCIO DO BLOCO INVARIANTE (NUNCA ALTERAR EM SESSÃO ATIVA) -->
# PROTOCOLO DE GOVERNANÇA AGÊNTICA – NÍVEL INDUSTRIAL

## DIRETIVAS DE COMUNICAÇÃO (DENSIDADE DE SHANNON)
1. Idioma obrigatório: Português do Brasil (PT-BR).
2. Estilo de resposta: Conciso, telegráfico, sem saudações e sem preâmbulos.
3. Pensamento interno (<thinking>): Estilo Caveman (abreviado, foco em fatos).

## DIRETIVAS DE ENGENHARIA (LOCALIDADE DE CONTEXTO)
1. REGRA INVIOLÁVEL: Use grep/ripgrep para localizar linhas antes de ler arquivos inteiros.
2. Nunca execute leituras superiores a 100 linhas sem autorização explícita.
3. Sempre execute os testes automatizados antes de reportar conclusão de tarefas.
<!-- FIM DO BLOCO INVARIANTE -->
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Reescreva um prompt vago usando o princípio de Invariância de Prefixo — mantenha o início estático e isole a parte dinâmica
- ☐ Meça a “densidade de Shannon” de um prompt que você usa com frequência: quantas palavras são ruído vs. informação técnica real?
- ☐ Aplique o princípio de Localidade de Contexto em uma tarefa real: use grep antes de ler qualquer arquivo inteiro
- ☐ Crie o seu CLAUDE.md invariante com pelo menos 3 regras fixas que não mudam entre sessões

⑦ Armadilhas

- (a completar)

Passo 6 — A Constituição Mestre: As 18 Regras Sagradas

Estágio: Painel operacional · **Origem:** Cap. 6 — A Constituição Mestre: As 18 Regras Sagradas

① Objetivo do passo

Detalhar cada uma das 18 regras com exemplos práticos.

② Pré-requisito

Passo 5 concluído

③ Entregas

- `.governance/CONSTITUTION.md`
- `CLAUDE.md`

④ Execução

Checklist Prático para Incorporação no seu Projeto

CONSTITUIÇÃO MESTRE DE GOVERNANÇA AGÊNTICA (18 REGRAS)

Você é um Agente de Engenharia subordinado ao Engenheiro Agêntico.

Você deve obedecer incondicionalmente às 18 Regras Sagradas:

1. Idioma PT-BR.
2. Densidade de Shannon (sem prosa).
3. Pensamento Caveman interno.
4. Links clicáveis para arquivos.
5. Evidências reais com Exit Code 0.
6. Confirmação prévia para ações destrutivas.

7. Grep antes de read.
8. Edição cirúrgica em blocos.
9. Imutabilidade do arquivo de governança.
10. Preservação de testes existentes.
11. Structured Outputs com schema.
12. Atomicidade (uma tarefa por vez).
13. Bloqueio de comandos perigosos.
14. Isolamento em worktrees.
15. Zero arquivos temporários no root.
16. Proibido salvar segredos no Git.
17. Circuit Breaker aos 3 erros repetidos.
18. Auditoria obrigatória de 6 gates.

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Selecione a regra da Constituição que teria evitado o maior erro que você já cometeu com IA e escreva por que ela é crítica
- ☐ Audite uma resposta antiga de um chat seu com IA e verifique quantas das 18 regras foram violadas pelo assistente
- ☐ Crie o arquivo `.governance/CONSTITUTION.md` no seu projeto com as 18 regras e referencie-o no CLAUDE.md
- ☐ Teste a regra R5 (Transparência de Evidências): peça ao agente para exibir o log real com Exit Code 0 antes de declarar sucesso

⑦ Armadilhas

- (a completar)

Passo 7 — O Motor de Economia Severa de Tokens

Estágio: Painel operacional · **Origem:** Cap. 7 — O Motor de Economia Severa de Tokens

① Objetivo do passo

Ensinar as 5 skills fundamentais com exemplos de redução de custo.

② Pré-requisito

Passo 6 concluído

③ Entregas

- (a completar)

④ Execução

Template Prático de Diretiva para Economia Severa

DIRETIVAS DE ECONOMIA SEVERA DE TOKENS (SISTEMA ARSENAL)

1. ****Caveman Thinking Obrigatório****:

- No bloco <thinking>, use apenas frases telegráficas e abreviações.
- Proibido repetir o prompt do usuário no pensamento interno.
- Máximo de 3 a 5 linhas de raciocínio para tarefas comuns.

2. ****Carregamento Tardio de Habilidades (Lazy Skills)****:

- Não carregue manuais de ferramentas até que a tarefa exija explicitamente.
- Ao precisar de especialidades, leia o arquivo .governance/skills/<nome_skill>/SKILL.md.

3. ****Truncamento de Saídas de Terminal****:

- Ao executar comandos que gerem mais de 50 linhas de saída, inspecione apenas o tail do log.

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Converta um raciocínio prolixo seu para o formato Caveman Thinking e compare o custo estimado de tokens
- ☐ Estruture a pasta .governance/skills/ no seu projeto com pelo menos 1 skill carregada sob demanda (Lazy Loading)
- ☐ Configure o truncamento de logs no seu ambiente: capture apenas as 20 primeiras e 20 últimas linhas de erros longos
- ☐ Meça o custo de tokens de uma sessão antes e depois de aplicar as técnicas de economia severa

⑦ Armadilhas

- (a completar)

Passo 8 — Implementação e Réplica da Camada 1

Estágio: Painel operacional · **Origem:** Cap. 8 — Implementação e Réplica da Camada 1

① Objetivo do passo

Guia passo a passo para implementar a Camada TELA.

② Pré-requisito

Passo 7 concluído

③ Entregas

- `setup_camada1.py`

④ Execução

Script Universal de Inicialização da Camada 1 (`setup_camada1.py`)

```
#!/usr/bin/env python3
# setup_camada1.py – Inicializador Universal da Camada 1
import os
import sys
from pathlib import Path

CONSTITUICAO_TEXTO = """# GOVERNANÇA AGÊNTICA – NÍVEL INDUSTRIAL (18 REGRAS)

## COMUNICAÇÃO (SHANNON DENSITY MAX)
1. Idioma: Português do Brasil (PT-BR).
2. Estilo: Direto, conciso, sem saudações ou preâmbulos.
3. Raciocínio interno (<thinking>): Estilo Caveman telegráfico.
4. Use links markdown clicáveis para arquivos de código.
5. Transparência: Apresente logs reais e Exit Code 0 em testes.
6. Ações destrutivas exigem consentimento prévio do operador.

## ENGENHARIA & CONTEXTO
7. Grep antes de read: Proibido carregar arquivos inteiros sem busca cirúrgica.
8. Edição cirúrgica: Substitua apenas blocos específicos de texto.
9. Imutabilidade: Nunca altere este arquivo durante uma sessão ativa.
10. Preservação de testes existentes: Proibido apagar asserções para forçar sucesso.
11. Structured Outputs com contratos JSON Schema rígidos.
12. Atomicidade: Um objetivo e uma responsabilidade por turno.

## HIGIENE & SEGURANÇA
13. Bloqueio estrito de comandos perigosos sem sandbox.
14. Isolamento em Git Worktrees para tarefas concorrentes.
15. Zero poluição: Não crie arquivos temporários na raiz do projeto.
16. Detecção ativa de segredos: Proibido salvar chaves de API no repositório.
17. Circuit Breaker: Pause aos 3 erros repetidos consecutivos.
18. Auditoria obrigatória de 6 gates de pre-commit antes de declarar entrega.
"""

def instalar_camada1():
    print("=== [CAMADA 1] Instalando Governança e Diretivas de Contexto ===")

    # 1. Criar pastas
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute o script `setup_camada1.py` no seu projeto e verifique se `CLAUDE.md`, `.governance/CONSTITUTION.md` e `AGENTS.md` foram criados

- ☐ Teste a reação do agente: faça uma pergunta simples e observe se ele responde em PT-BR direto, sem saudações (Densidade de Shannon)
- ☐ Configure os hardlinks/junctions para que o mesmo arquivo de regras atenda a múltiplas IDEs sem duplicação
- ☐ Rode a auditoria automatizada da Camada 1 e confirme que o KV-Cache não foi corrompido

⑦ Armadilhas

- (a completar)

Passo 9 — Os 3 Princípios Universais do HARNESS

Estágio: Circuito de segurança · **Origem:** Cap. 9 — Os 3 Princípios Universais do HARNESS

① Objetivo do passo

Ensinar os fundamentos de Harness & Loop Engineering.

② Pré-requisito

Passo 8 concluído

③ Entregas

- `.harness/settings.json`

④ Execução

Arquivo de Configuração do Harness (`.harness/settings.json`)

```
{
  "harness_governance": {
    "version": "2.0",
    "circuit_breakers": {
      "max_turns_per_task": 15,
      "command_timeout_seconds": 60,
      "max_consecutive_errors": 3
    },
    "command_blacklist": [
      "rm -rf /",
      "rm -rf *",
      "mkfs",
      "dd if=",
      "git push --force",
      "git reset --hard origin/main",
      ":(){ :|:& };;"
    ],
    "sandbox_policy": {
```

```
    "require_git_worktree": true,  
    "auto_rollback_on_failure": true  
  },  
  "hooks_enabled": {  
    "pre_tool_call": true,  
    "post_tool_call": true,  
    "pre_commit_gates": true  
  }  
}
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Crie a pasta `.harness/` no seu projeto e salve o arquivo `settings.json` com os limites de segurança adequados (turnos, timeout, blacklist)
- ☐ Teste o disjuntor: tente executar um comando da blacklist (ex: `rm -rf /`) e observe o bloqueio
- ☐ Crie um Git Worktree isolado e execute uma tarefa de teste dentro dele, comprovando que a branch principal fica intacta
- ☐ Configure um hardlink/junction para o arquivo de governança e verifique que a atualização propaga para todas as ferramentas

⑦ Armadilhas

- (a completar)

Passo 10 — Configuração Industrial: Circuit Breakers e Sandbox

Estágio: Circuito de segurança · **Origem:** Cap. 10 — Configuração Industrial: Circuit Breakers e Sandbox

① Objetivo do passo

Ensinar a configurar `settings.json` com proteções industriais.

② Pré-requisito

Passo 9 concluído

③ Entregas

- `circuit_breaker.py`

④ Execução

Implementação Prática do Interceptor de Segurança (circuit_breaker.py)

```
#!/usr/bin/env python3
# circuit_breaker.py – Guardião de Comandos e Disjuntor da Camada 2
import sys
import re

COMMAND_BLACKLIST = [
    r"rm\s+--rf\s+[/~]",          # Tentativa de apagar raiz ou home
    r"mkfs",                      # Formatação de disco
    r"git\s+push\s+.*--force",    # Commit forçado destrutivo
    r"drop\s+database",           # Deleção de banco
    r":\(\)\{\ :\\:;&\};;",        # Fork bomb
]

def validar_comando(comando: str, turnos_atuais: int, limite_turnos: int = 15) -> bool:
    # 1. Checagem do Disjuntor de Turnos
    if turnos_atuais > limite_turnos:
        print(f"[DISJUNTOR DESARMADO] Limite de {limite_turnos} turnos atingido. Pausando agente.")
        return False

    # 2. Checagem da Blacklist de Comandos
    for padrao in COMMAND_BLACKLIST:
        if re.search(padrao, comando, re.IGNORECASE):
            print(f"[COMANDO BLOQUEADO] Padrão proibido detectado: {padrao}")
            return False

    print(f"[HARNESS SEGURO] Comando autorizado para execução: {comando[:40]}...")
    return True

if __name__ == "__main__":
    cmd_teste = sys.argv[1] if len(sys.argv) > 1 else "npm test"
    if not validar_comando(cmd_teste, turnos_atuais=5):
        sys.exit(1)
    sys.exit(0)
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute o script `circuit_breaker.py` passando o comando `"rm -rf /"` e observe o bloqueio instantâneo com Exit Code 1
- ☐ Crie um Git Worktree manual com `git worktree add ../minha-sandbox -b teste-seguro` e comprove que é uma cópia isolada
- ☐ Configure o `.harness/settings.json` com os limites de turnos, timeout e blacklist adequados ao seu projeto
- ☐ Teste o loop de auto-cura por AST: force uma falha de teste e observe o agente corrigindo apenas a função defeituosa

⑦ Armadilhas

- (a completar)

Passo 11 — O Guarda-Costas do Git: Pre-Commit com 6 Gates

Estágio: Circuito de segurança · **Origem:** Cap. 11 — O Guarda-Costas do Git: Pre-Commit com 6 Gates

① Objetivo do passo

Ensinar a implementar pre-commit hook com 6 gates.

② Pré-requisito

Passo 10 concluído

③ Entregas

- `.harness/hooks/pre-commit`
- `.git/hooks/pre-commit`

④ Execução

O Script Oficial do Pre-Commit Hook (`.harness/hooks/pre-commit`)

```
#!/usr/bin/env bash
# pre-commit — O Guarda-Costas do Git com 6 Gates
set -e

echo "=== [PRE-COMMIT] Iniciando Inspeção dos 6 Gates de Integridade ==="

# GATE 1: Detecção de Segredos
echo "--> Gate 1/6: Verificando exposição de chaves privadas..."
if grep -rE "sk-ant-[a-zA-Z0-9_-]{20,}|ghp_[a-zA-Z0-9]{20,}" --exclude-dir=".git" .; then
    echo "[FALHA GATE 1] Segredo detectado no código! Abortando commit."
    exit 1
fi
echo "    [OK] Nenhum segredo exposto."

# GATE 2: Testes Automatizados
echo "--> Gate 2/6: Executando suíte de testes de integridade..."
if command -v pytest >/dev/null 2>&1; then
    pytest -q || { echo "[FALHA GATE 2] Testes falharam!"; exit 1; }
fi
echo "    [OK] Testes automatizados passaram com Exit Code 0."

# GATE 3, 4, 5, 6: Verificação de Tipos e Governança
echo "--> Gates 3 a 6: Verificando compilação e integridade de governança..."
if [ ! -f "CLAUDE.md" ]; then
    echo "[FALHA GATE 6] Arquivo de governança CLAUDE.md foi apagado!"; exit 1;
fi
```

```
echo "      [OK] Governança e tipagem verificadas."

echo "=== [SUCESSO] Todos os Gates Aprovados! Commit Liberado. ==="
exit 0
```

⑤ Verificação / Gate

```
pytest -q || { echo "[FALHA GATE 2] Testes falharam!"; exit 1; }
```

⑥ Feito quando...

- ❑ Crie o arquivo `teste_segredo.txt` com uma chave `sk-ant-...` falsa e confirme que o Gate 1 bloqueia o commit
- ❑ Instale o script `pre-commit` em `.git/hooks/pre-commit` com `chmod +x` e rode um commit de teste
- ❑ Adicione um teste unitário que falha de propósito e observe o Gate 2 abortando o commit com Exit Code 1
- ❑ Verifique que o Gate 6 aborta o commit se o arquivo `CLAUDE.md` for apagado ou adulterado

⑦ Armadilhas

- (a completar)

Passo 12 — Implementação e Réplica da Camada 2

Estágio: Circuito de segurança · **Origem:** Cap. 12 — Implementação e Réplica da Camada 2

① Objetivo do passo

Guia passo a passo para implementar a Camada HARNESS.

② Pré-requisito

Passo 11 concluído

③ Entregas

- `setup_camada2.py`

④ Execução

Script Universal de Instalação da Camada 2 (`setup_camada2.py`)

```
#!/usr/bin/env python3
# setup_camada2.py – Instalador Industrial da Camada 2 (Harness)
```

```

import os
import sys
import shutil
from pathlib import Path

SETTINGS_JSON = """{
    "harness": {
        "version": "2.0",
        "circuit_breakers": {
            "max_turns": 15,
            "timeout_seconds": 60
        },
        "blocked_commands": [
            "rm -rf /", "mkfs", "git push --force", "drop database"
        ],
        "gates": {
            "secret_detection": true,
            "unit_tests": true,
            "governance_parity": true
        }
    }
}"""

def instalar_camada2():
    print("=== [CAMADA 2] Instalando Harness, Circuit Breakers e Hooks ===")

    # 1. Criar pasta .harness
    dir_harness = Path(".harness")
    dir_hooks = dir_harness / "hooks"
    dir_hooks.mkdir(parents=True, exist_ok=True)

    # 2. Gravar settings.json
    (dir_harness / "settings.json").write_text(SETTINGS_JSON, encoding="utf-8")
    print(" [OK] Arquivo .harness/settings.json configurado.")

    # 3. Configurar hook de pre-commit no Git se a pasta .git existir
    git_hooks = Path(".git") / "hooks"
    if git_hooks.exists():
        hook_file = git_hooks / "pre-commit"
        hook_conteudo = """#!/usr/bin/env bash

set -e
echo "[PRE-COMMIT HARNESS] Executando validação de segurança..."
if grep -rE "sk-ant-[a-zA-Z0-9_-]{20,}" --exclude-dir=".git" .; then
    echo "[ERRO] Chave de API detectada! Abortando commit."
    exit 1
fi
echo "[PRE-COMMIT HARNESS] Aprovado com Exit Code 0."
exit 0
"""
        hook_file.write_text(hook_conteudo, encoding="utf-8")
        try:
            os.chmod(hook_file, 0o755)
        except Exception:

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `python setup_camada2.py` no seu projeto e confirme que `.harness/settings.json` e o hook de pre-commit foram criados
- ☐ Monte uma Topologia Hierárquica: um coordenador despachando 2 subagentes em worktrees separados
- ☐ Simule uma Topologia Gauntlet: um agente implementa uma função e outro tenta quebrá-la com casos de borda
- ☐ Configure os setup-links (junctions) para que `.governance/` seja compartilhado entre duas ferramentas

⑦ Armadilhas

- (a completar)

Passo 13 — Os 3 Princípios Universais do LLM

Estágio: Roteador cognitivo · **Origem:** Cap. 13 — Os 3 Princípios Universais do LLM

① Objetivo do passo

Ensinar os fundamentos de Model Layer & Semantic Routing.

② Pré-requisito

Passo 12 concluído

③ Entregas

- `semantic_router.py`

④ Execução

Exemplo de Roteador Semântico Simples em Python (`semantic_router.py`)

```
#!/usr/bin/env python3
# semantic_router.py – Roteador Cognitivo da Camada 3
import sys

# Matriz de Tiers
TIER_1_FAST = "gemini-2.0-flash"      # Quase gratuito / Rápido
TIER_2_DEV  = "claude-3-7-sonnet"    # Desenvolvimento equilibrado
TIER_3_DEEP = "claude-3-7-thinking"  # Raciocínio arquitetural pesado

PALAVRAS_CHAVE_COMPLEXAS = ["arquitetura", "seguranca", "refatorar sistema", "algoritmo",
                             "otimizar sql"]

def rotear_tarefa(descricao_tarefa: str) -> str:
    texto = descricao_tarefa.lower()
```

```

# Tarefas Críticas (Tier 3)
if any(p in texto for p in PALAVRAS_CHAVE_COMPLEXAS):
    print(f"[ROTEADOR C3] Tarefa Crítica detectada -> Roteando para TIER 3
    ({TIER_3_DEEP})")
    return TIER_3_DEEP

# Tarefas de Codificação Padrão (Tier 2)
if any(p in texto for p in ["criar funcao", "escrever teste", "adicionar endpoint",
"consertar"]):
    print(f"[ROTEADOR C3] Desenvolvimento Padrão -> Roteando para TIER 2
    ({TIER_2_DEV})")
    return TIER_2_DEV

# Tarefas Mecânicas (Tier 1)
print(f"[ROTEADOR C3] Tarefa Mecânica Simples -> Roteando para TIER 1
    ({TIER_1_FAST})")
    return TIER_1_FAST

if __name__ == "__main__":
    prompt = sys.argv[1] if len(sys.argv) > 1 else "Formatar o arquivo de logs"
    modelo_escolhido = rotear_tarefa(prompt)
    print(f"Modelo alocado: {modelo_escolhido}")

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `semantic_router.py` com as frases “Formatar o arquivo de logs”, “criar funcao de login” e “desenhar arquitetura de segurança” e observe a alocação de tiers
- ☐ Classifique 5 tarefas reais do seu dia a dia entre Tier 1, Tier 2 e Tier 3
- ☐ Defina um JSON Schema para uma resposta de API e teste que o modelo devolve exatamente a estrutura esperada
- ☐ Simule uma falha do provedor principal e verifique o fallback automático para o provedor secundário

⑦ Armadilhas

- (a completar)

Passo 14 — A Matriz de 3 Tiers de Modelos

Estágio: Roteador cognitivo · **Origem:** Cap. 14 — A Matriz de 3 Tiers de Modelos

① Objetivo do passo

Ensinar o roteamento inteligente entre 3 tiers de modelos.

② Pré-requisito

Passo 13 concluído

③ Entregas

- `.router/tiers.json`

④ Execução

Configuração Declarativa da Matriz de Tiers (`.router/tiers.json`)

```
{
  "tiers_matrix": {
    "tier_1_fast": {
      "primary": "gemini-2.0-flash",
      "fallback": "claude-3-5-haiku",
      "max_tokens_budget": 1000,
      "temperature": 0.1
    },
    "tier_2_standard": {
      "primary": "claude-3-7-sonnet",
      "fallback": "deepseek-chat",
      "max_tokens_budget": 4000,
      "temperature": 0.2
    },
    "tier_3_reasoning": {
      "primary": "claude-3-7-sonnet-thinking",
      "fallback": "o3-mini",
      "thinking_budget": 8000,
      "temperature": 1.0
    }
  }
}
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Crie o arquivo `.router/tiers.json` com os 3 tiers e insira os modelos que você possui configurados
- ☐ Classifique 5 tarefas reais entre Tier 1, Tier 2 e Tier 3 e justifique cada escolha
- ☐ Simule a Regra de Escalação: force uma falha dupla no Tier 2 e observe a escalação automática para o Tier 3
- ☐ Estime o custo de um projeto seu usando a estratégia monolítica vs a estratégia dos 3 Tiers

⑦ Armadilhas

- (a completar)

Passo 15 — Contratos Tipados e Registro Declarativo

Estágio: Roteador cognitivo · **Origem:** Cap. 15 — Contratos Tipados e Registro Declarativo

① Objetivo do passo

Ensinar JSON Schemas e registro declarativo de tipos.

② Pré-requisito

Passo 14 concluído

③ Entregas

- (a completar)

④ Execução

Exemplo Prático de Contrato Tipado em Python com Pydantic

```
#!/usr/bin/env python3
# contrato_tipado.py – Exemplo de Structured Output com Pydantic
from pydantic import BaseModel, Field
from typing import List, Literal

# 1. Definição do Contrato Formal
class ItemRelatorio(BaseModel):
    modulo: str = Field(description="Nome do módulo auditado")
    status: Literal["aprovado", "reprovado", "alerta"] = Field(description="Estado de conformidade")
    erros_encontrados: int = Field(default=0, ge=0, description="Quantidade de bugs detectados")
    recomendacao: str = Field(description="Ação técnica recomendada")

class RelatorioAuditoria(BaseModel):
    projeto: str
    versao: str
    total_modulos: int
    itens: List[ItemRelatorio]

# 2. Exibição do JSON Schema gerado automaticamente
if __name__ == "__main__":
    print("=== JSON SCHEMA DO CONTRATO DE AUDITORIA ===")
    print(RelatorioAuditoria.schema_json(indent=2))
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Defina um JSON Schema para cadastrar um livro com `titulo` (texto), `paginas` (inteiro) e `categoria` (apenas “tecnologia”, “ficção” ou “negócios”)
- ☐ Execute `python contrato_tipado.py` e observe o schema matemático gerado pela biblioteca Pydantic
- ☐ Teste a validação: envie um JSON com campo inválido (ex: `paginas` como texto) e confirme que o Pydantic rejeita
- ☐ Crie um contrato tipado para uma resposta de API sua e verifique que o modelo devolve exatamente a estrutura esperada

⑦ Armadilhas

- (a completar)

Passo 16 — Implementação e Réplica da Camada 3

Estágio: Roteador cognitivo · **Origem:** Cap. 16 — Implementação e Réplica da Camada 3

① Objetivo do passo

Guia passo a passo para implementar o roteador LLM.

② Pré-requisito

Passo 15 concluído

③ Entregas

- `setup_camada3.py`

④ Execução

O Script Oficial do Roteador Cognitivo (`setup_camada3.py`)

```
#!/usr/bin/env python3
# setup_camada3.py – Instalador Industrial da Camada 3 (Roteador Cognitivo)
import os
import sys
from pathlib import Path

ROUTER_CONFIG = """{
  "routing_strategy": "pareto_80_20",
  "providers": {
    "tier_1_fast": {
      "model": "gemini-2.0-flash",
      "timeout": 15,
      "max_retries": 2
    },
    "tier_2_standard": {
      "model": "claude-3-7-sonnet",
```

```

        "timeout": 30,
        "max_retries": 2
    },
    "tier_3_reasoning": {
        "model": "claude-3-7-sonnet-thinking",
        "timeout": 60,
        "max_retries": 1
    }
},
"fallbacks": {
    "claude-3-7-sonnet": "deepseek-chat",
    "gemini-2.0-flash": "claude-3-5-haiku"
}
}"""

def instalar_camada3():
    print("=== [CAMADA 3] Instalando Motor Cognitivo e Roteador Semântico ===")

    # 1. Criar pasta .router
    dir_router = Path(".router")
    dir_schemas = dir_router / "schemas"
    dir_schemas.mkdir(parents=True, exist_ok=True)

    # 2. Gravar models_config.json
    (dir_router / "models_config.json").write_text(ROUTER_CONFIG, encoding="utf-8")
    print(" [OK] Arquivo .router/models_config.json configurado com estratégia Pareto 80/20.")

    # 3. Gravar schema exemplo
    schema_exemplo = """{
"$schema": "http://json-schema.org/draft-07/schema#",
"title": "RespostaPadraoAgente",
"type": "object",
"properties": {
    "sucesso": { "type": "boolean" },
    "resumo": { "type": "string" },
    "arquivos_modificados": { "type": "array", "items": { "type": "string" } }
},
"required": ["sucesso", "resumo", "arquivos_modificados"]
}"""

    (dir_schemas / "resp

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `python setup_camada3.py` e confirme que `.router/models_config.json` e `.router/schemas/resposta_padrao.json` foram criados
- ☐ Adicione o seu modelo secundário favorito na lista de fallbacks do `models_config.json`
- ☐ Simule uma queda do provedor principal e meça o tempo de chaveamento para o fallback
- ☐ Roteie 5 tarefas reais e verifique se cada uma foi alocada ao tier de menor custo adequado

⑦ Armadilhas

- (a completar)

Passo 17 — Os 3 Princípios Universais de TOOLS

Estágio: Usina determinística · **Origem:** Cap. 17 — Os 3 Princípios Universais de TOOLS

① Objetivo do passo

Ensinar os fundamentos de MCP Servers & Determinismo Mecânico.

② Pré-requisito

Passo 16 concluído

③ Entregas

- `tool_idempotente.py`

④ Execução

Exemplo de Ferramenta Idempotente em Python (`tool_idempotente.py`)

```
#!/usr/bin/env python3
# tool_idempotente.py – Exemplo de Ferramenta Segura da Camada 4
import hashlib
from pathlib import Path

def garantir_configuracao_no_arquivo(caminho_arquivo: str, chave: str, valor: str) -> dict:
    """
    # 1. Se o arquivo não existir, cria e grava
    if not arq.exists():
        arq.write_text(linha_alvo, encoding="utf-8")
        hash_final = hashlib.md5(arq.read_bytes()).hexdigest()
        return {"status": "criado", "md5": hash_final, "exit_code": 0}

    conteudo_atual = arq.read_text(encoding="utf-8")

    # 2. Idempotência: se a linha já existir exatamente igual, não duplica
    if linha_alvo in conteudo_atual:
        hash_final = hashlib.md5(arq.read_bytes()).hexdigest()
        return {"status": "ja_existia_inalterado", "md5": hash_final, "exit_code": 0}

    # 3. Adiciona a linha de forma limpa
    arq.write_text(conteudo_atual + linha_alvo, encoding="utf-8")
    hash_final = hashlib.md5(arq.read_bytes()).hexdigest()
    return {"status": "atualizado", "md5": hash_final, "exit_code": 0}
```

```
if __name__ == "__main__":
    resultado = garantir_configuracao_no_arquivo("app.env", "DATABASE_PORT", "5432")
    print(f"Resultado da Ferramenta: {resultado}")
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `python tool_idempotente.py` três vezes seguidas e comprove que `DATABASE_PORT=5432` foi gravado apenas uma vez
- ☐ Calcule o hash MD5 do seu `CLAUDE.md` com `hashlib` e registre o valor
- ☐ Refatore uma função “canivete suíço” sua em 3 ferramentas atômicas com papel único
- ☐ Teste a paridade de integridade: altere um byte de um arquivo crítico e confirme que o hash diverge

⑦ Armadilhas

- (a completar)

Passo 18 — O Banco de Estado Persistente: SQLite e a Esteira

Estágio: Usina determinística · **Origem:** Cap. 18 — O Banco de Estado Persistente: SQLite e a Esteira

① Objetivo do passo

Ensinar SQLite como banco de estado para agentes IA.

② Pré-requisito

Passo 17 concluído

③ Entregas

- `state_manager.py`

④ Execução

Módulo Completo de Rastreamento de Estado em Python (`state_manager.py`)

```
#!/usr/bin/env python3
# state_manager.py – Gerenciador de Estado Persistente da Camada 4
import sqlite3
import json
from pathlib import Path
```

```

DB_PATH = Path(".tools/state_tracker.db")

def inicializar_banco():
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    # Ativar modo WAL para alta concorrência
    cursor.execute("PRAGMA journal_mode=WAL;")

    # Criar tabela de tarefas
    cursor.execute("""
CREATE TABLE IF NOT EXISTS tarefas_esteira (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT NOT NULL,
    agente_responsavel TEXT NOT NULL,
    status TEXT CHECK(status IN ('pendente', 'em_andamento', 'concluido', 'falha'))
DEFAULT 'pendente',
    arquivos_modificados TEXT,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
""")
    conn.commit()
    conn.close()
    print(" [OK] Banco de Estado SQLite WAL inicializado em .tools/state_tracker.db.")

def registrar_tarefa(titulo: str, agente: str) -> int:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("INSERT INTO tarefas_esteira (titulo, agente_responsavel, status)
VALUES (?, ?, 'em_andamento')", (titulo, agente))
    task_id = cursor.lastrowid
    conn.commit()
    conn.close()
    return task_id

def concluir_tarefa(task_id: int, arquivos: list):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("UPDATE tarefas_esteira SET status = 'concluido', arquivos_modificados
= ? WHERE id = ?", (json.dumps(arquivos), task_id))
    conn.commit()

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `python state_manager.py` e confirme que `.tools/state_tracker.db` foi criado com a tabela `tarefas_esteira`
- ☐ Consulte o banco com um visualizador de SQLite e verifique as tarefas registradas
- ☐ Desenhe o esquema de uma tarefa do seu cotidiano e liste os campos adicionais úteis na tabela

- ☐ Teste a persistência: registre uma tarefa, feche e reabra o programa, e confirme que o estado foi restaurado

⑦ Armadilhas

- (a completar)

Passo 19 — Servidores MCP e a Usina de Scripts Determinísticos

Estágio: Usina determinística · **Origem:** Cap. 19 — Servidores MCP e a Usina de Scripts Determinísticos

① Objetivo do passo

Ensinar arquitetura MCP e scripts determinísticos.

② Pré-requisito

Passo 18 concluído

③ Entregas

- `servidor_mcp_simples.py`

④ Execução

Criando seu Primeiro Servidor MCP em Python (`servidor_mcp_simples.py`)

```
#!/usr/bin/env python3
# servidor_mcp_simples.py — Servidor MCP Oficial da Camada 4
import sys
import json

def processar_requisicao(req_json: str) -> str:
    try:
        dados = json.loads(req_json)
        metodo = dados.get("method")

        # 1. Descoberta de Ferramentas (tools/list)
        if metodo == "tools/list":
            return json.dumps({
                "tools": [
                    {
                        "name": "calcular_soma",
                        "description": "Calcula a soma determinística de dois números
inteiros",
                        "inputSchema": {
                            "type": "object",
                            "properties": {
                                "a": { "type": "integer" },
```

```

        "b": { "type": "integer" }
    },
    "required": ["a", "b"]
}
    ]
})

# 2. Execução de Ferramenta (tools/call)
elif metodo == "tools/call":
    params = dados.get("params", {})
    args = params.get("arguments", {})
    resultado = args.get("a", 0) + args.get("b", 0)
    return json.dumps({
        "content": [{"type": "text", "text": str(resultado)}],
        "isError": False
    })

except Exception as e:
    return json.dumps({"isError": True, "error": str(e)})

return json.dumps({"isError": True, "error": "Método desconhecido"})

if __name__ == "__main__":
    # Teste

```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `python servidor_mcp_simples.py` alterando a chamada de teste para `{"method": "tools/list"}` e observe a descoberta de ferramentas
- ☐ Adicione ao script uma segunda ferramenta `multiplicar_numeros` com o seu schema correspondente
- ☐ Teste a execução tipada: chame `calcular_soma` com argumentos válidos e inválidos e confirme a resposta estruturada
- ☐ Crie um script determinístico seu (ex: `renderizar-diagramas.py`) e comprove que a mesma entrada produz sempre a mesma saída

⑦ Armadilhas

- (a completar)

Passo 20 — O Super-Auditor e o Manual de Montagem Universal

Estágio: Usina determinística · **Origem:** Cap. 20 — O Super-Auditor e o Manual de Montagem Universal

① Objetivo do passo

Super-Auditor e manual para novos e legados projetos.

② Pré-requisito

Passo 19 concluído

③ Entregas

- `super_auditor.py`

④ Execução

O Script Completo do Super-Auditor Universal (`super_auditor.py`)

```
#!/usr/bin/env python3
# super_auditor.py – Auditor Mestre das 4 Camadas da Fábrica Agêntica
import os
import sys
import json
from pathlib import Path

def auditar_quatro_camadas():
    print("=====")
    print("  SUPER-AUDITOR UNIVERSAL – TRATADO DAS 4 CAMADAS DA FÁBRICA  ")
    print("=====")

    score = 0
    relatorio = {}

    # 1. Auditoria da Camada 1 (Contexto & Governança)
    print("
--> [1/4] Auditando Camada 1: Contexto & Diretivas...")
    c1_ok = Path("CLAUDE.md").exists() or Path(".governance/CONSTITUTION.md").exists()
    relatorio["camada_1_contexto"] = "APROVADO" if c1_ok else "REPROVADO"
    if c1_ok:
        score += 25
        print("  [OK] Arquivo de Governança Invariante ativo.")
    else:
        print("  [ALERTA] Ausência de CLAUDE.md ou CONSTITUTION.md!")

    # 2. Auditoria da Camada 2 (Harness & Segurança)
    print("
--> [2/4] Auditando Camada 2: Harness & Execução...")
    c2_ok = Path(".harness/settings.json").exists() or Path(".git").exists()
    relatorio["camada_2_harness"] = "APROVADO" if c2_ok else "REPROVADO"
    if c2_ok:
        score += 25
        print("  [OK] Disjuntores e controle de versão ativos.")
    else:
        print("  [ALERTA] Ausência de .harness/settings.json ou Git!")

    # 3. Auditoria da Camada 3 (Motor Cognitivo)
    print("
--> [3/4] Auditando Camada 3: Motor Cognitivo & Roteamento...")
    c3_ok = Path(".router/models_config.json").exists() or Path(".router"
```

⑤ Verificação / Gate

(a completar)

⑥ Feito quando...

- ☐ Execute `python super_auditor.py` na raiz do seu projeto e registre a pontuação obtida
- ☐ Se a pontuação for inferior a 100, execute os instaladores das camadas correspondentes (`setup_camada1.py` , `setup_camada2.py` , `setup_camada3.py` , `state_manager.py`) até atingir 100/100
- ☐ Lance o seu primeiro agente autônomo em um worktree isolado com a Constituição Mestre ativa
- ☐ Documente o pipeline de ponta a ponta do seu projeto passando pelas 4 camadas até o Super-Auditor

⑦ Armadilhas

- (a completar)

Checklist Mestre

Passo 1 — O Contexto Real de Origem: O Projeto Arsenal Open Source

- ☐ Verifique se seu projeto possui um arquivo de governança invariante (CLAUDE.md ou equivalente — se não existir, crie um com 3 regras mínimas)
- ☐ Escreva uma lista das 4 dores que você já enfrentou ao usar IA para programar (custo, amnésia, alucinação, lock-in)
- ☐ Documente o seu fluxo atual de interação com agentes de IA e identifique ao menos 1 ponto onde a Metriz de Transposição Universal poderia ser aplicada
- ☐ Crie um esboço em papel ou Markdown de como sua Central de Comando seria organizada (4 camadas)

Passo 2 — O Dicionário do Iniciante: Glossário Descomplicado

- ☐ Defina em suas próprias palavras o que são LLM, MCP, Token e SQLite — escreva como se explicasse para um colega de trabalho leigo
- ☐ Crie um glossário pessoal com pelo menos 10 termos do capítulo que você mais usa ou pretende usar no dia a dia
- ☐ Identifique 3 termos que ainda lhe causam confusão e pesquise cada um até dominar o conceito
- ☐ Explique para alguém leigo o que é um “agente de IA” usando apenas analogias do mundo real

Passo 3 — A Crise do Desenvolvimento com IA: Os 4 Problemas Catastróficos

- ☐ Analise uma conversa recente com IA e identifique se houve amnésia (Lost in the Middle) — o agente esqueceu algo que foi dito anteriormente?
- ☐ Calcule aproximadamente quanto gastou em tokens na última semana de uso de IA e compare com o que gastaria com Invariância de Prefixo ativa
- ☐ Liste 3 códigos gerados por IA que precisaram de correção manual depois de o agente declarar “sucesso”
- ☐ Documente 1 caso concreto de alucinação que já presenciou: o agente afirmou que algo estava pronto, mas não estava

Passo 4 — Visão Geral das 4 Camadas: A Arquitetura Completa

- ☐ Esboce em papel ou Markdown como as 4 camadas se conectam no seu projeto atual — use o modelo da Fórmula 1 como referência visual
- ☐ Identifique em qual das 4 camadas seu projeto está mais fraco hoje e Justifique por quê
- ☐ Crie um plano de implementação para a Camada 1 (Contexto & Diretivas) com pelo menos 3 ações concretas
- ☐ Mapeie as ferramentas que você usa hoje e classifique cada uma em qual camada ela opera

Passo 5 — Os 3 Princípios Universais da TELA

- ☐ Reescreva um prompt vago usando o princípio de Invariância de Prefixo — mantenha o início estático e isole a parte dinâmica
- ☐ Meça a “densidade de Shannon” de um prompt que você usa com frequência: quantas palavras são ruído vs. informação técnica real?

- ☐ Aplique o princípio de Localidade de Contexto em uma tarefa real: use grep antes de ler qualquer arquivo inteiro
- ☐ Crie o seu CLAUDE.md invariante com pelo menos 3 regras fixas que não mudam entre sessões

Passo 6 — A Constituição Mestre: As 18 Regras Sagradas

- ☐ Selecione a regra da Constituição que teria evitado o maior erro que você já cometeu com IA e escreva por que ela é crítica
- ☐ Audite uma resposta antiga de um chat seu com IA e verifique quantas das 18 regras foram violadas pelo assistente
- ☐ Crie o arquivo `.governance/CONSTITUTION.md` no seu projeto com as 18 regras e referencie-o no CLAUDE.md
- ☐ Teste a regra R5 (Transparência de Evidências): peça ao agente para exibir o log real com Exit Code 0 antes de declarar sucesso

Passo 7 — O Motor de Economia Severa de Tokens

- ☐ Converta um raciocínio prolixo seu para o formato Caveman Thinking e compare o custo estimado de tokens
- ☐ Estruture a pasta `.governance/skills/` no seu projeto com pelo menos 1 skill carregada sob demanda (Lazy Loading)
- ☐ Configure o truncamento de logs no seu ambiente: capture apenas as 20 primeiras e 20 últimas linhas de erros longos
- ☐ Meça o custo de tokens de uma sessão antes e depois de aplicar as técnicas de economia severa

Passo 8 — Implementação e Réplica da Camada 1

- ☐ Execute o script `setup_camada1.py` no seu projeto e verifique se CLAUDE.md, `.governance/CONSTITUTION.md` e AGENTS.md foram criados
- ☐ Teste a reação do agente: faça uma pergunta simples e observe se ele responde em PT-BR direto, sem saudações (Densidade de Shannon)
- ☐ Configure os hardlinks/junctions para que o mesmo arquivo de regras atenda a múltiplas IDEs sem duplicação
- ☐ Rode a auditoria automatizada da Camada 1 e confirme que o KV-Cache não foi corrompido

Passo 9 — Os 3 Princípios Universais do HARNESS

- ☐ Crie a pasta `.harness/` no seu projeto e salve o arquivo `settings.json` com os limites de segurança adequados (turnos, timeout, blacklist)
- ☐ Teste o disjuntor: tente executar um comando da blacklist (ex: `rm -rf /`) e observe o bloqueio
- ☐ Crie um Git Worktree isolado e execute uma tarefa de teste dentro dele, comprovando que a branch principal fica intacta
- ☐ Configure um hardlink/junction para o arquivo de governança e verifique que a atualização propaga para todas as ferramentas

Passo 10 — Configuração Industrial: Circuit Breakers e Sandbox

- ☐ Execute o script `circuit_breaker.py` passando o comando `"rm -rf /"` e observe o bloqueio instantâneo com Exit Code 1
- ☐ Crie um Git Worktree manual com `git worktree add ../minha-sandbox -b teste-seguro` e comprove que é uma cópia isolada

- ☐ Configure o `.harness/settings.json` com os limites de turnos, timeout e blacklist adequados ao seu projeto
- ☐ Teste o loop de auto-cura por AST: force uma falha de teste e observe o agente corrigindo apenas a função defeituosa

Passo 11 — O Guarda-Costas do Git: Pre-Commit com 6 Gates

- ☐ Crie o arquivo `teste_segredo.txt` com uma chave `sk-ant-...` falsa e confirme que o Gate 1 bloqueia o commit
- ☐ Instale o script `pre-commit` em `.git/hooks/pre-commit` com `chmod +x` e rode um commit de teste
- ☐ Adicione um teste unitário que falha de propósito e observe o Gate 2 abortando o commit com Exit Code 1
- ☐ Verifique que o Gate 6 aborta o commit se o arquivo `CLAUDE.md` for apagado ou adulterado

Passo 12 — Implementação e Réplica da Camada 2

- ☐ Execute `python setup_camada2.py` no seu projeto e confirme que `.harness/settings.json` e o hook de pre-commit foram criados
- ☐ Monte uma Topologia Hierárquica: um coordenador despachando 2 subagentes em worktrees separados
- ☐ Simule uma Topologia Gauntlet: um agente implementa uma função e outro tenta quebrá-la com casos de borda
- ☐ Configure os setup-links (junctions) para que `.governance/` seja compartilhado entre duas ferramentas

Passo 13 — Os 3 Princípios Universais do LLM

- ☐ Execute `semantic_router.py` com as frases “Formatar o arquivo de logs”, “criar funcao de login” e “desenhar arquitetura de seguranca” e observe a alocação de tiers
- ☐ Classifique 5 tarefas reais do seu dia a dia entre Tier 1, Tier 2 e Tier 3
- ☐ Defina um JSON Schema para uma resposta de API e teste que o modelo devolve exatamente a estrutura esperada
- ☐ Simule uma falha do provedor principal e verifique o fallback automático para o provedor secundário

Passo 14 — A Matriz de 3 Tiers de Modelos

- ☐ Crie o arquivo `.router/tiers.json` com os 3 tiers e insira os modelos que você possui configurados
- ☐ Classifique 5 tarefas reais entre Tier 1, Tier 2 e Tier 3 e justifique cada escolha
- ☐ Simule a Regra de Escalação: force uma falha dupla no Tier 2 e observe a escalação automática para o Tier 3
- ☐ Estime o custo de um projeto seu usando a estratégia monolítica vs a estratégia dos 3 Tiers

Passo 15 — Contratos Tipados e Registro Declarativo

- ☐ Defina um JSON Schema para cadastrar um livro com `titulo` (texto), `paginas` (inteiro) e `categoria` (apenas “tecnologia”, “ficção” ou “negócios”)
- ☐ Execute `python contrato_tipado.py` e observe o schema matemático gerado pela biblioteca Pydantic
- ☐ Teste a validação: envie um JSON com campo inválido (ex: `paginas` como texto) e confirme que o Pydantic rejeita

- ☐ Crie um contrato tipado para uma resposta de API sua e verifique que o modelo devolve exatamente a estrutura esperada

Passo 16 — Implementação e Réplica da Camada 3

- ☐ Execute `python setup_camada3.py` e confirme que `.router/models_config.json` e `.router/schemas/resposta_padrao.json` foram criados
- ☐ Adicione o seu modelo secundário favorito na lista de fallbacks do `models_config.json`
- ☐ Simule uma queda do provedor principal e meça o tempo de chaveamento para o fallback
- ☐ Roteie 5 tarefas reais e verifique se cada uma foi alocada ao tier de menor custo adequado

Passo 17 — Os 3 Princípios Universais de TOOLS

- ☐ Execute `python tool_idempotente.py` três vezes seguidas e comprove que `DATABASE_PORT=5432` foi gravado apenas uma vez
- ☐ Calcule o hash MD5 do seu `CLAUDE.md` com `hashlib` e registre o valor
- ☐ Refatore uma função “canivete suíço” sua em 3 ferramentas atômicas com papel único
- ☐ Teste a paridade de integridade: altere um byte de um arquivo crítico e confirme que o hash diverge

Passo 18 — O Banco de Estado Persistente: SQLite e a Esteira

- ☐ Execute `python state_manager.py` e confirme que `.tools/state_tracker.db` foi criado com a tabela `tarefas_esteira`
- ☐ Consulte o banco com um visualizador de SQLite e verifique as tarefas registradas
- ☐ Desenhe o esquema de uma tarefa do seu cotidiano e liste os campos adicionais úteis na tabela
- ☐ Teste a persistência: registre uma tarefa, feche e reabra o programa, e confirme que o estado foi restaurado

Passo 19 — Servidores MCP e a Usina de Scripts Determinísticos

- ☐ Execute `python servidor_mcp_simples.py` alterando a chamada de teste para `{"method": "tools/list"}` e observe a descoberta de ferramentas
- ☐ Adicione ao script uma segunda ferramenta `multiplicar_numeros` com o seu schema correspondente
- ☐ Teste a execução tipada: chame `calcular_soma` com argumentos válidos e inválidos e confirme a resposta estruturada
- ☐ Crie um script determinístico seu (ex: `renderizar-diagramas.py`) e comprove que a mesma entrada produz sempre a mesma saída

Passo 20 — O Super-Auditor e o Manual de Montagem Universal

- ☐ Execute `python super_auditor.py` na raiz do seu projeto e registre a pontuação obtida
- ☐ Se a pontuação for inferior a 100, execute os instaladores das camadas correspondentes (`setup_camada1.py`, `setup_camada2.py`, `setup_camada3.py`, `state_manager.py`) até atingir 100/100
- ☐ Lance o seu primeiro agente autônomo em um worktree isolado com a Constituição Mestre ativa
- ☐ Documente o pipeline de ponta a ponta do seu projeto passando pelas 4 camadas até o Super-Auditor